Algorithmique & Architectures parallèles

Résumé de cours complet — introduction, fondements, performances, architectures, réseaux d'interconnexion, OpenMP

1 Introduction au calcul parallèle
2 Fondements : formes et sources de parallélisme
3 Grain, degré, T_{opt} et P_{opt}
4 Évaluation des performances
5 Architectures parallèles (Flynn, mémoire, clusters…)
6 Multi-cœurs, GPU et CUDA
7 Réseaux d'interconnexion
8 Programmation avec OpenMP
9 Fiche formules & questions de révision

1 · Introduction au calcul parallèle

DÉFINITION

Le **calcul parallèle** consiste à utiliser **plusieurs unités de calcul** (processeurs autonomes) capables de **communiquer** et de **coopérer** dans le but d'**accélérer** la résolution d'**un seul problème**. Autres noms : calcul parallèle et distribué, calcul à haute performance (HPC), *parallel computing*.

	Algorithme séquentiel	Algorithme parallèle
Principe	Suite d'instructions exécutées l'une après l'autre	Le même algorithme exécuté par plusieurs unités de calcul
Dépendance	Chaque instruction dépend de la précédente	Nécessite communication, coopération, synchronisation
Complexité	Somme des opérations	Max (des branches exécutées en parallèle)

- On donne à chaque processeur une tâche ; ils échangent des données. Une tâche ne doit pas casser celle d'un autre : il faut **ordonnancer** les tâches (rôle du système d'exploitation).
- Question clé : les processeurs partagent-ils la même mémoire, ou passent-ils par un réseau ? ⇒ **avant d'écrire l'algorithme, il faut connaître l'architecture**.
- Chaîne du cours : **Architecture parallèle → Algorithmique parallèle → Programmes parallèles**. Chaque problème a un **potentiel de parallélisme** (nul ou fort).

Pourquoi le parallélisme est devenu indispensable

- Les applications « **3T** » demandent 1 Téraflops, 1 Téraoctet de mémoire centrale et 1 Téraoctet/s de bande passante. On parle maintenant d'applications « **3P** » (péta = 10^{15} = 10^3 téra). *flops* = *floating-point operations per second* (opérations sur les réels).
- Ces applications ne sont réalisables qu'avec un **parallélisme massif** ⇒ HPC (classement mondial : top500.org).
- Grands challenges : modélisation et simulation (conception aéronautique, prévision météo), ingénierie (génie nucléaire), calculs symboliques (reconnaissance de la parole, compréhension du langage naturel, vision par ordinateur).

Temps séquentiel vs temps parallèle

$$T_{séq} = T_{traitement} \quad (1 \text{ facteur : le processeur})$$

$$T_{par} = T_{traitement} + T_{communication}$$ (2 facteurs : processeurs + réseau d'interconnexion)

2 · Fondements : formes et sources de parallélisme

Formes de parallélisme

Implicite	Extrait automatiquement par le compilateur ou le matériel (pipeline, superscalaire). Le programmeur n'intervient pas.
Explicite	Le développeur parallélise lui-même (threads, MPI, OpenMP…).

Deux approches : **séquentiel → parallèle** (adapter un programme existant) ou **parallèle dès la conception** (facile sur du code simple, difficile sur du code complexe).

QUAND PEUT-ON PARALLÉLISER ?

Si S_1 et S_2 sont **dépendantes** (ex. accès à la même case mémoire : $x := f(...)$; $y := x + ...$), elles s'exécutent en **séquentiel**. Exemple du cours : S1 → S2 → S3 → S4 où S3/S4 réutilisent x, y. Si dans S3 et S4 on remplace x par une autre variable (x'), on obtient deux chaînes indépendantes **S1→S2 ‖ S3→S4**.

Les trois sources de parallélisme

Source	Principe	Caractéristiques
1. Données	Le **même calcul** appliqué en parallèle à des **données différentes**. Exploite la régularité des données (tableaux, listes, matrices).	Travail répétitif, faible complexité, facile à découper. Difficulté : la distribution des données.
2. Tâches	Des **calculs différents** sur des **données différentes**, en même temps.	On ne peut pas répartir au hasard : on étudie les dépendances. But : minimiser temps et ressources.
3. Flux (pipeline)	Travail **à la chaîne** : chaque processeur est spécialiste d'une étape ; chaque donnée subit une séquence de traitements en pipeline.	Moins courant. Ex. : station de lavage (laver, rincer, sécher).

Combiner plusieurs modes = **parallélisme mixte**.

Exemple — parallélisme de données

pour i de 1 à n : A(i) ← A(i) + 4. Cas idéal p = n : $T_1 = n$, $T_n = 1 \Rightarrow S_n = n$, $E_n = 1$.

Exemple — parallélisme de tâches (1 u.t. par tâche)

t_1 : a ← a+2 t_2 : b ← b−4 t_3 : d ← d+9 t_4 : c ← a×b−d ⇒ **$t_1 ‖ t_2 ‖ t_3 \Rightarrow t_4$**

$T_1 = 4$, avec p = 3 : $T_3 = 2 \Rightarrow S_3 = 2$, $E_3 = 2/3 \approx 66$ % (un processeur inactif pendant la 2e unité de temps).

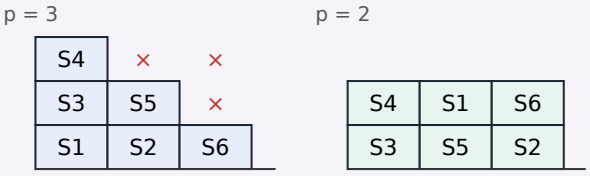
EXEMPLE DU COURS — GRAPHE + DIAGRAMME DE GANTT

S1 : a = 2 S2 : b = a×2 S3 : c = 4
S4 : d = 5 S5 : e = c−d S6 : f = a+e

Dépendances : S1→S2, S1→S6, S3→S5, S4→S5, S5→S6.

3 processeurs : $T_{séq} = 6$, $T_3 = 3 \Rightarrow S_3 = 2$, $E_3 = 2/3$ (66 %).
2 processeurs : $T_2 = 3 \Rightarrow S_2 = 2$, $E_2 = 1$ (100 %).

Même accélération avec moins de processeurs ⇒ meilleure efficacité.



axe horizontal = temps (u.t.), lignes = processeurs
× = processeur inactif

Exemple — parallélisme de flux (pipeline)

pour i de 1 à n : C(i) ← [(A(i) + 2) × B(i)] / 3 → 3 étapes θ_1 (+), θ_2 (×), θ_3 (/) sur P_1, P_2, P_3. Pendant que P_2 traite la donnée i, P_1 traite déjà i+1.

	T_1	T_p	S_p	E_p
3 étapes	3n	n + 2	3n/(n+2) → 3	n/(n+2) → 1
p étapes (généralisation)	p·n	n + p − 1	p·n/(n+p−1) ≈ p si p ≪ n	n/(n+p−1) ≈ 1 si p ≪ n

Les p − 1 unités supplémentaires = **latence du pipeline** (remplissage/vidange).

3 · Grain, degré de parallélisme, T_{opt} et P_{opt}

GRAIN (GRANULARITÉ)
Taille moyenne des tâches élémentaires issues de la parallélisation (en nombre d'instructions ou en temps d'exécution). Son choix est **fortement lié à l'architecture** :

- communication lente ⇒ **gros grain**
- communication rapide ⇒ **grain fin**

DEGRÉ DE PARALLÉLISME
Nombre de sous-tâches exécutées **simultanément**. Indique le nombre de processeurs utilisables. Il varie pendant l'exécution ⇒ on considère $\mathbf{D_{max}}$, $\mathbf{D_{moy}}$, $\mathbf{D_{min}}$, évalués statiquement (compilation) ou dynamiquement (exécution).

COMPROMIS DU GRAIN
Grain trop fin ⇒ surcoût de communication et de synchronisation. Grain trop gros ⇒ déséquilibre de charge, processeurs inactifs.

EXEMPLE — DEGRÉS
S0 ; S1 ∥ S2 ∥ S3 ; S4 ∥ S5 ∥ S6 ∥ S7 ; S8 ∥ S9 ; S10 → degrés successifs 1, 3, 4, 2, 1
$D_{max} = 4$, $D_{min} = 1$, $D_{moy} = (1+3+4+2+1)/5 = 11/5 = 2{,}2$

Temps et nombre de processeurs optimaux

- $\mathbf{T_{opt}}$: temps optimal d'exécution parallèle = somme des durées des tâches sur le **chemin critique** (le plus long chemin) du **graphe de dépendance** (DAG : sommets = tâches, arcs = dépendances). On peut le déterminer **indépendamment du nombre de processeurs**.
- $\mathbf{P_{opt}}$: nombre **minimal** de processeurs permettant d'atteindre T_{opt}. C'est un optimum global : il ne dépend pas de la représentation choisie.

- Toujours : $T_p \geq T_{opt} \Rightarrow S_p \leq T_1/T_{opt}$, quel que soit p.

EXERCICE 1 — $P_{OPT} = D_{MAX}$

$t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$ (mêmes tâches qu'en §2)

$D_{min} = 1$, $D_{max} = 3$, $D_{moy} = 2$

$T_{opt} = 2$, $P_{opt} = 3 = D_{max}$

EXERCICE 2 — $P_{OPT} < D_{MAX}$

On ajoute t_5 : e ← e+1 (indépendante) : $t_1 \parallel t_2 \parallel t_3 \parallel t_5 \Rightarrow t_4$

Version naïve : $D_{max} = 4$, $T_{opt} = 2$ avec p = 4. Mais on peut placer t_5 en parallèle de t_4 : T = 2 avec 3 processeurs ⇒ **$P_{opt} = 3 < D_{max} = 4$**.

4 · Évaluation des performances

Notations : n = taille du problème, $T_1(n)$ = temps séquentiel (1 processeur), $T_p(n)$ = temps parallèle (p processeurs).

Accélération (speedup)

$$S_p(n) = \frac{T_1(n)}{T_p(n)}$$

- $S_p < 1$: ralentissement (mauvaise parallélisation)
- $1 < S_p \leq p$: cas normal
- $S_p > p$: accélération **super-linéaire**

Efficacité

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T_1(n)}{p \cdot T_p(n)}$$

Taux d'utilisation des processeurs. **$E_p = 1$** = idéal (tous occupés en permanence). L'efficacité est maximale quand la charge est **équilibrée**.

POURQUOI UNE ACCÉLÉRATION SUPER-LINÉAIRE ?

T_1 = temps de swap (échanges mémoire/cache) + temps de traitement. Si les données ne tiennent pas dans le cache d'un seul processeur (ex. matrice $10^6 \times 10^6$), il y a beaucoup de swaps. En parallèle, chaque processeur ne manipule qu'une partie qui tient en cache : le swap disparaît et T_p = traitement/p < T_1/p ⇒ $S_p > p$.

Loi d'Amdahl

Si une fraction *f* du programme est strictement séquentielle :

$$S_p \leq \frac{1}{f + (1-f)/p} \rightarrow \frac{1}{f} \quad \text{quand } p \rightarrow \infty$$

Ex. : f = 10 % ⇒ l'accélération ne dépassera **jamais 10**, quel que soit le nombre de processeurs.

5 · Architectures parallèles

Monotâche (les tâches s'enchaînent) → **multitâche** (entrelacées sur un processeur) → **parallèle** (vraiment simultanées sur plusieurs processeurs). Le domaine s'est fortement développé depuis 1985. Deux grandes catégories de machines : **à mémoire partagée** (dont vectorielles) et **à mémoire distribuée**. Aujourd'hui, les **clusters** dominent.

Classification de Flynn (1966)

Deux flux : **flux d'instructions** (séquence d'instructions exécutées) et **flux de données** (séquence de données appelées par les instructions).

	1 flux de données (SD)	Plusieurs flux de données (MD)
1 flux d'instructions (SI)	**SISD** — machines séquentielles	**SIMD** — machines vectorielles ($a_i = b_i + c_i$ pour tout i)
Plusieurs flux d'instructions (MI)	**MISD** — *aucune machine réelle*	**MIMD** — multiprocesseurs

SPMD (Single Program Multiple Data) : variante de MIMD, même programme sur des données différentes. Ordre : **SIMD < SPMD < MIMD**. Flynn reste simpliste pour classer finement.

	SIMD	MIMD
Principe	Un frontal + processeurs identiques synchronisés : même opération au même moment, données différentes	Chaque processeur exécute son propre code sur ses données
Exécution	Synchrone	Asynchrone, indépendante
Processeurs	Spécifiques (petite mémoire locale + réseau)	Ceux du marché
Usage	Calcul régulier (traitement d'images)	Usage général — classe la plus répandue
Exemples / avenir	Cray-1, NEC SX-5, CM-1 — en voie de disparition	Sous-classes : mémoire partagée / distribuée

Mémoire partagée vs mémoire distribuée

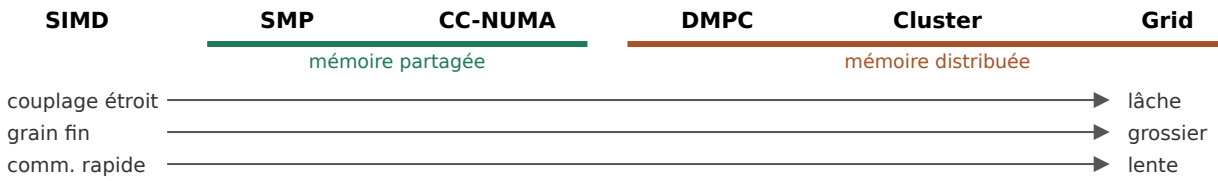
	Mémoire partagée (SMP)	Mémoire distribuée
Mémoire globale	Oui, partagée par tous (2 à 64 processeurs)	Non : chaque processeur a la sienne
Communication	Implicite, via la mémoire	Explicite, par messages sur le réseau
Programmation	Facile	Plus difficile
Extensibilité	Faible, coût élevé	Forte — seules à offrir de très hautes performances
Exemples	PC bi-processeur, Cray SV1, SGI Origin3000	Transputers, Cray T3E, IBM SP3

	UMA	NUMA (cc-NUMA)
Temps d'accès	Identique partout	Dépend de la distance processeur–module
Construction	Une mémoire commune (= SMP)	Plusieurs SMP reliés
Extensibilité	Limitée	Extensible

- **CLUMPS** (Clusters of SMP) : mémoire partagée dans chaque SMP, distribuée entre les SMP (ex. IBM SP3, nœuds de 16 processeurs).
- **Hybride** : mémoire physiquement distribuée, programmée comme partagée (ex. SGI Origin2000).

P PROCESSEURS ≠ P FOIS PLUS RAPIDE
Les processeurs doivent échanger des données ; le coût des communications est très élevé. **Optimiser les mouvements de données est la clé** de la performance en mémoire distribuée.

Classement de Leopold (2001)



De gauche à droite, les machines s'éloignent (de la puce jusqu'à Internet) : couplage lâche, communication lente ⇒ seulement de gros grains.

Système parallèle vs système distribué

Système parallèle	Système distribué
Éléments de calcul qui communiquent et coopèrent pour résoudre rapidement de grands problèmes.	Processeurs autonomes, sans mémoire primaire partagée, qui coopèrent par messages sur un réseau.

Parallèle non distribué : vectorielles, SMP. Parallèle et distribué : stations sur réseau local, CLUMPS. Distribué non parallèle : stations longue distance. Ni l'un ni l'autre : PC isolés.

Clusters et grilles

	Cluster (grappe)	Grille (Grid)
Définition	Collection de nœuds **faiblement couplés** reliés par un réseau d'interconnexion	Collection de **clusters hétérogènes**, dispersés géographiquement
Réseau	LAN rapide (Fast Ethernet, Myrinet) : haute bande passante, faible latence	WAN / Internet
Hétérogénéité	Homogène ou hétérogène	Forte ; middleware qui masque l'architecture
Apparition	Années 90 (alternative économique aux supercalculateurs)	Années 2000 — image de Foster (1998) : le réseau électrique
Exemples	Tera-10 (Bull, CEA), TSUBAME (NEC)	GRID5000 (France), TeraGrid (USA)

Époque	1970s	1980s	1990s	2000s	Après 2000
Parallélisme	Processeur	Machine	LAN	WAN	Puce
Architecture	Vectorielle	SMP / MPP	Cluster	Grid	Multi-cœur
Programmation	Vector Fortran	C*, C-Linda, Occam	PVM, MPI, HPF	MPI, OpenMP	?

6 · Multi-cœurs, GPU et CUDA

Processeurs multi-cœurs

Au moins **deux cœurs** sur la même puce. On augmente la puissance **sans augmenter la fréquence**, avec moins d'énergie et de chaleur que les multiprocesseurs.

Trois types : caches distincts, cache commun, « Quad-Core » Intel (= 2 puces double-cœur).

GPU

Circuit de la carte graphique, structure **hautement parallèle**. Libère le CPU des calculs d'affichage.

ATTENTION
Les programmes doivent être repensés, sinon un seul cœur travaille.

	CPU	GPU
Transistors	Caches, contrôle	Beaucoup plus d'ALU
Bande passante	Plus faible	Plus élevée
Rôle	Parties séquentielles	Calcul intensif

Mémoire graphique partagée (part de la RAM) ou dédiée (sur la carte). Carte dédiée = puissante ; IGP intégré = moins cher, économe.

CUDA (NVIDIA)

- API, extension du langage C : interface de programmation + runtime + bibliothèque mathématique.
- Programme = **code série sur le CPU (host)** + **kernels sur le GPU (device)** exécutés par des threads parallèles.
- Un kernel s'exécute comme une **grille de blocs de threads**. Les threads d'un bloc coopèrent, se synchronisent et partagent la **shared memory**. Les blocs s'exécutent comme des **CTA** sur les cœurs **SM**.

7 · Réseaux d'interconnexion

Critères de classement : mode d'opération (synchrone/asynchrone), stratégie de contrôle (centralisée/décentralisée), commutation (circuit/paquet), topologie (**statique** / **dynamique** : connexions établies à la volée).

Machines à mémoire partagée (réseaux dynamiques)

Le réseau relie les bus de données et d'adresses des processeurs à n'importe quel module mémoire. Bus de données de même largeur d ; adresse = couple (b, a) : b = banc, a = case ⇒ largeur b + a côté processeur, a côté mémoire.

	Bus (0 étage)	Crossbar (1 étage)	Multi-étages
Connexion	Tous sur un bus, réservé jusqu'à l'arrivée de l'information	Chaque P relié à chaque banc par des switchs (graphe complet)	Étages de boîtes d'échange (pas de lignes directes)
Parallélisme	Aucun : si P1 envoie à M3, P2 vers M1 attend	Envois simultanés vers des bancs différents	Simultané s'il n'y a pas de conflit dans une boîte
Conflit	Exclusion mutuelle	Exclusion mutuelle sur le même banc	Arbitrage dans la boîte

BOÎTE D'ÉCHANGE
2 entrées, 2 sorties, deux positions : **straight** (e1→s1, e2→s2) ou **exchange** (e1→s2, e2→s1). Deux processeurs qui demandent la **même position** passent simultanément ; sinon il faut un arbitrage.

Machines à mémoire distribuée (passage de messages)

Message = entête (destinataire, expéditeur, infos) + corps. Le réseau est un graphe G(X, U) : **n** = nombre de processeurs, **m** = nombre de liens, **Δ** = degré maximal, **D** = diamètre (plus grande distance entre deux nœuds), **K** = connectivité (nombre de chemins différents). Petit D = messages

rapides ; petits m, Δ = réseau moins cher ; **K ≥ 2** ⇒ communications parallèles possibles et tolérance aux pannes ; K = 1 ⇒ pas de parallélisme.

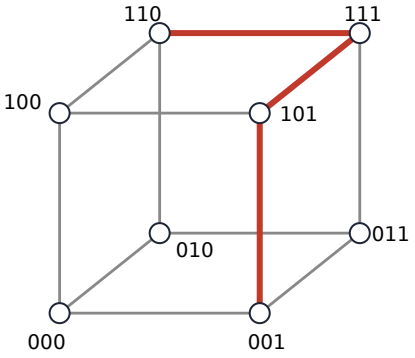
Topologie	n	m	D	Δ	K
Chaîne linéaire	p	$p - 1$	$p - 1$	2	1
Anneau	p	p	$E(p/2)$	2	2
Grille 2D $q_1 \times q_2$	$q_1 q_2$	$2q_1q_2 - (q_1+q_2)$	$q_1 + q_2 - 2$	4	≥ 2
Grille 2D torique	$q_1 q_2$	$2q_1q_2$	$E(q_1/2) + E(q_2/2)$	4	≥ 2
Grille 3D	—	—	—	—	≥ 3
Arbre binaire (h niveaux)	$2^h - 1$	$2^h - 2$	$2(h - 1)$	3	1
Hypercube de dimension d	2^d	$d \cdot 2^{d-1}$	d	d	d

E = partie entière. Pour l'hypercube, K = d est déduit de l'exemple du 3-cube (3 chemins parallèles).

Hypercube

- Récurrence : un d-cube = deux (d−1)-cubes superposés, chaque sommet relié à son correspondant : $n_d = 2n_{d-1}$, $m_d = 2m_{d-1} + n_{d-1}$, $\Delta_d = \Delta_{d-1} + 1$, $D_d = D_{d-1} + 1$.
- Numérotation binaire sur d bits ; deux nœuds sont **voisins s'ils diffèrent d'un seul bit**.
- Distance = **distance de Hamming** (nombre de bits différents).

CHEMINS DE 001 À 110 (3-CUBE)
3 bits différents ⇒ distance 3 et 3 chemins disjoints :
001→101→111→110 · 001→011→010→110 · 001→000→100→110



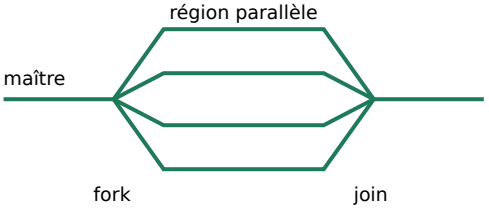
3-cube : en rouge, 001 → 101 → 111 → 110

8 · Programmation parallèle avec OpenMP

OpenMP (Open MultiProcessing) : API à base de **directives** pour la programmation parallèle en **mémoire partagée** (C, C++, Fortran). Approche la plus simple : paralléliser les boucles.

Modèle fork-join

- Un seul thread au départ : le **thread maître**.
- **Fork** : début de région parallèle, création d'une équipe de threads.
- **Join** : fin de région, **barrière implicite** ; seul le maître continue.
- Thread = « processus léger » ; les threads d'un processus partagent sa mémoire virtuelle.



Mise en œuvre

```
#include <omp.h>
gcc -fopenmp -o mon_prog mon_source.c
export OMP_NUM_THREADS=2          // par défaut : nb threads = nb CPU
double t0 = omp_get_wtime();  /* travail */  printf("%f\n", omp_get_wtime() - t0);
```

Fonction	Rôle
`omp_get_num_threads()`	Nombre de threads en cours
`omp_get_thread_num()`	Rang du thread courant
`omp_set_num_threads(nb)`	Fixe le nombre de threads
`omp_in_parallel()`	1 si dans une région parallèle

```
#pragma omp parallel
{
  int rang = omp_get_thread_num();
  printf("Hello du thread %d\n", rang);
  if (rang == 0)
    printf("Total : %d\n",
omp_get_num_threads());
}
```

Statut des variables

`shared`	Par défaut, toutes les variables sont partagées (sauf les compteurs de boucle)
`private`	Privée à chaque thread, **non initialisée**
`firstprivate`	Privée, initialisée avec la valeur d'origine
`lastprivate`	Privée ; la variable d'origine reçoit la valeur de la dernière itération
`threadprivate`	Privée, persiste d'une région parallèle à l'autre (initialisée par `copyin`)

Clauses de `parallel` : `num_threads(n)`, `if(expr)` (séquentiel si expr = 0), `default(shared|none)`, `copyin`, `copyprivate` (diffuse après un `single`, incompatible avec `nowait`), `nowait` (supprime la barrière implicite).

Synchronisation

`master`	Exécuté par le maître (rang 0) seul ; les autres continuent
`single`	Exécuté par un seul thread (le premier arrivé) ; les autres attendent
`barrier`	Tous les threads s'attendent
`critical`	Un seul thread à la fois dans la section
`atomic`	Mise à jour atomique d'une variable (++, --, op. logique)

Boucles, réduction, ordonnancement, sections

```
#pragma omp parallel for reduction(+ : somme) schedule(dynamic, 4)
for (i = 0; i < N; i++) somme += f(i);       // i est privée par défaut

#pragma omp parallel for ordered
for (i = 0; i < N; i++) { ... #pragma omp ordered  printf("%d\n", i); }

#pragma omp sections
{ #pragma omp section { tache_A(); }   #pragma omp section { tache_B(); } }
```

schedule	Comportement
`static`	Par défaut : N/p itérations par thread ; avec une taille de bloc, distribution circulaire (round-robin)
`dynamic`	Un thread qui termine prend le bloc suivant — adapté aux itérations de coûts inégaux
`guided`	Comme dynamic, mais la taille des blocs décroît avec les itérations restantes (x = N/p, y = (N−x)/p, …)
`runtime` / `auto`	Choix à l'exécution / laissé au compilateur

Exemple du cours : initialisation d'une matrice 9000×9000 (a[i][j] = i*j) avec `parallel for num_threads(2) private(j)` : gain proche de 2, limité par la création des threads et les accès mémoire (malloc).

9 · Fiche formules & questions de révision

Notion	Formule / règle
Accélération	$S_p = T_1 / T_p$ (normal : $1 < S_p \leq p$; super-linéaire : $S_p > p$)
Efficacité	$E_p = S_p / p = T_1 / (p \cdot T_p)$ (idéal = 1)
Amdahl	$S_p \leq 1 / (f + (1-f)/p) \to 1/f$
Pipeline (p étapes, n données)	$T_1 = pn$, $T_p = n+p-1$, $S_p \approx p$, $E_p \approx 1$ si $p \ll n$
Degré moyen	D_{moy} = moyenne des degrés de chaque étape
T_{opt} / P_{opt}	T_{opt} = chemin critique du DAG ; P_{opt} = min de processeurs pour T_{opt} ; $P_{opt} \leq D_{max}$
Borne	$T_p \geq T_{opt} \Rightarrow S_p \leq T_1/T_{opt}$
Hypercube	$n = 2^d$, $m = d \cdot 2^{d-1}$, $D = \Delta = d$, distance = Hamming

Questions de révision

Q1. Quelles sont les 4 classes de Flynn ? Laquelle n'a aucune machine réelle ? — SISD, SIMD, MISD, MIMD ; c'est MISD.

Q2. Quelles sont les trois sources de parallélisme ? — Données (même calcul, données différentes), tâches (calculs différents), flux (pipeline). Leur combinaison = parallélisme mixte.

Q3. Différence implicite / explicite ? — Implicite : extrait par le compilateur/matériel. Explicite : écrit par le développeur.

Q4. 4 tâches $t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$ sur 3 processeurs : S et E ? — $T_1 = 4$, $T_3 = 2$, $S = 2$, $E = 2/3$.

Q5. Pipeline à 3 étages sur n = 100 données : S_3 ? — 300/102 ≈ 2,94 ; $E_3 = 100/102 \approx 0{,}98$.

Q6. Pourquoi peut-on avoir $S_p > p$? — Les données découpées tiennent dans les caches : plus de swap.

Q7. f = 5 % séquentiel : accélération maximale ? — 1/0,05 = 20.

Q8. Peut-on avoir $P_{opt} < D_{max}$? — Oui : en déplaçant une tâche indépendante sur un créneau libre (ex. t_5 à côté de t_4), T_{opt} est atteint avec moins de processeurs.

Q9. UMA vs NUMA ? — UMA : même temps d'accès partout. NUMA : dépend de la distance ; plusieurs SMP reliés ; extensible.

Q10. Cluster vs grille ? — Cluster : nœuds faiblement couplés sur LAN rapide. Grille : clusters hétérogènes dispersés sur WAN/Internet, rendus transparents par le middleware.

Q11. Quand y a-t-il conflit dans une boîte d'échange ? — Quand deux processeurs demandent des positions différentes (straight vs exchange) : arbitrage.

Q12. Grille 2D 4×5 : n, m, D ? — n = 20 ; m = 40 − 9 = 31 ; D = 7.

Q13. Hypercube de dimension 4 : n, m, D, Δ ? — 16 ; 32 ; 4 ; 4. Distance entre 0110 et 1011 : 3.

Q14. Différence `private` / `firstprivate` ? — private n'est pas initialisée ; firstprivate prend la valeur d'origine.

Q15. `master` vs `single` ? — master : thread 0, pas de barrière ; single : un thread quelconque, les autres attendent à la fin.

Q16. Quel `schedule` pour des itérations de coûts très inégaux ? — `dynamic` (ou `guided`).